

TCG Chaos Inventory — Product Strategy Memo

Forxia Industries Corp. · Narrative memo (Amazon 6-pager format) · August 2026 · Confidential

About this format. Amazon's "6-pager" is a narrative memo (~6 pages, single-spaced prose) used in place of slide decks: Introduction, Goals, Tenets, State of the Business, Lessons Learned, Strategic Priorities, plus an unlimited Appendix for tables and data. Readers consume it silently before discussion. This document follows that structure for TCG Chaos Inventory.

1. Introduction

Trading card game shops lose margin and staff time at the boundary between *bulk* and *sellable* inventory. A trade-in arrives as an unsorted pile; staff must decide what to brick for position-picking, what to sort for individual listing, and how to keep channels from overselling stock that does not exist. Most shop software assumes everything is already sorted into SKUs. That assumption breaks the moment a shop accepts a 3,000-card collection on a Saturday afternoon.

TCG Chaos Inventory, developed by Forxia Industries Corp., is a self-hosted shop operating system built for that reality. Its differentiator is deliberate support for **chaos blocks**—sealed, position-indexed bulk inventory—alongside a planned **sorted stock** mode for individually sellable singles. A physical card lives in exactly one mode; moving between them is explicit and audited. The product runs on the shop's own server (Docker + PostgreSQL). There is no platform commission and no vendor lock-in on inventory data.

Phases 1–4 of the product are largely complete: staging intake from ManaBox CSV, formalized blocks on shelves and bins, Mana Pool order import, and location-aware pick lists including hold, quarantine, and correction workflows. The strategic question for the next 12–18 months is whether to complete the path to full-shop parity—staff identity, sellable stock, autopricing, marketplace sync, counter POS, and buylist—without abandoning the bulk model that trade-in-heavy shops depend on.

2. Goals

Output metrics (what success looks like for a pilot shop):

- Reduce time from trade-in scan to pickable inventory by 50% versus manual spreadsheet workflows.
- Zero oversells on sorted stock once channel sync (Phase 9) is live.
- Staff can answer "where is this card?" for bulk (block + position) or singles (SKU) in under 30 seconds.

- Shop owner can export backup, restore, and operate without Forxia cloud dependency.

Input metrics (levers we control through engineering):

- Ship Phases 5–6 (pricing persistence, search, staff login, SKU ledger) within the next development cycle.
- Complete minimum viable full-shop parity (Phases 6–10) before pursuing parked items (consignment, kiosk, SaaS multi-tenant).
- Maintain dockerized regression tests for every money- or inventory-affecting workflow.

3. Tenets

These principles resolve trade-offs without escalating every decision:

- **Bulk honesty over fake SKUs.** Do not force every common into a singles row. Chaos blocks exist because unsorted inventory is real; the system must model it natively.
- **Self-hosted first.** The shop owns the database and backup. SaaS hosting is a later option, not the default architecture.
- **Audit before scale.** No POS, buylist, or channel sync without staff identity on every event. Unattributed money movement cannot be retrofitted.
- **CSV before API.** Marketplace CSV templates ship before live integrations—they cover every channel at low risk and survive API outages.
- **One promote bridge.** Cards move from bulk to sorted stock only through an explicit, logged promote action—never implicitly.
- **Speed over completeness on intake.** Scan externally, review in staging, commit once. Perfect in-app camera scan is valuable but not blocking for MVP parity.

4. State of the Business

As of August 2026, TCG Chaos Inventory is a working single-tenant deployment aimed at Magic: The Gathering shops with heavy trade-in volume. The application stack is Next.js 15, React 19, Prisma, and PostgreSQL 16, packaged in Docker. Domain logic lives in tested library modules; mutations write to an append-only inventory event log.

What works end-to-end today: ManaBox CSV upload into staging; review and formalize into numbered blocks with position index; block lifecycle and sealed bricks; Mana Pool listing CSV export per block; Mana Pool order import; pick lists sorted by warehouse location with pick/short/substitute, renumber, counter pick, hold, quarantine, and correction intake; block aging analytics; JSON backup and restore; activity feed.

What is incomplete or broken: Market prices fetched during import are not persisted at formalize, so dollar-based valuation reads zero—a defect blocking

analytics value, autopricing, and buylist. Card search by name is stubbed. There is no authentication; the activity log cannot name an actor. Sorted stock, channel sync, POS, and buylist are specified but unbuilt. The parity target is SortSwift-class capability (multi-channel, rule-based pricing, basic POS) while retaining chaos blocks as a first-class mode.

Market position: SortSwift and similar tools optimize for sorted singles and live quantity sync. We compete by serving shops whose workflow starts with unsorted bulk, then optionally promotes high-value cards to singles—without maintaining two disconnected systems.

5. Lessons Learned

Building Phases 1–4 validated several assumptions and overturned others. Staging as a commit point—review before formalize—was essential; shops distrust whole scans far more often than single-card errors, so undo-at-import-level mattered more than partial block surgery. Position-indexed intake proved necessary at scale: without pile order, pick lists become unrecoverable when a brick is wrong.

We learned that treating “partial” features as done is costly. Pick routing helpers existed before the pick UI shipped; price fetch existed before persistence—creating dashboards that imply value while showing \$0. A August 2026 audit corrected backlog statuses against code; sales literature now derives from the same source of truth.

Dependency order for parity is non-negotiable: identity (ACC) before money features; price persistence (V-005) before autopricing; SKU ledger before channel sync and POS. Skipping this sequence produces rework. Deferred items—manual in-app intake, camera scan first, consignment, kiosk—were correctly parked; they do not unblock revenue for pilot shops.

6. Strategic Priorities

Priority 1 — Finish the chaos-mode product (Phase 5). Ship pricing persistence so valuation and capital-in-inventory reports mean something. Deliver card search and global quantity. Harden inbound webhooks and cron authentication. Outcome: a shop can run bulk MTG operations credibly while we build sorted stock.

Priority 2 — Foundation for sellable inventory (Phase 6). Staff accounts, roles, and actor on every event. Stock item ledger with reservations and a browser. Sort-to-stock at intake and promote-from-block bridge with cost basis. Outcome: the dual inventory model is real in software, not only on architecture diagrams.

Priority 3 — Multi-game and intake review (Phase 7). Game registry and catalog providers; Pokémon as second game. Alternate printing picker and price overlay during staging review—most scan-parity value without camera ML first. Outcome: shops beyond MTG-only can pilot; review friction drops.

Priority 4 — Autopricing (Phase 8). Scheduled refresh, ordered rule engine, per-channel configuration, bulk reprice, and rule explanation ("why this price?"). Outcome: staff stop hand-editing thousands of rows.

Priority 5 — Channel sync (Phase 9). Channel registry, marketplace CSV templates, one live API channel, oversell guard, bidirectional quantity reconciliation, order ingestion, sync health. Outcome: sorted stock quantities stay truthful across channels.

Priority 6 — Counter and buylist (Phase 10). POS cart against live stock, store credit, tax and receipts, returns; internal buylist engine before customer-facing submission portal. Outcome: in-store trade-in and sales loop inside the same system as online channels.

Priority 7 — Unified fulfilment (Phase 11). Single order queue, fulfilment deduction, shipping labels and tracking, returns—after POS and channels produce orders. Outcome: one pick/ship workflow regardless of order source.

Parked until a pilot shop needs them commercially: consignment, advanced owner reporting, kiosk, events, native mobile, public API, multi-tenant SaaS (ADR-010 trigger-gated).

Appendix A — Feature summary (available today)

Area	Capabilities
Intake	ManaBox CSV, staging, block breakdown, position index, formalize, undo import, 5k+ cards/import
Inventory	Blocks, shelf/bin/location, lifecycle, move, seal, audit log
Picking	Mana Pool orders, location sort, pick/short/sub, hold, quarantine, counter pick, TCGplayer pullsheet
Analytics	Aging buckets, stale list (currency pending pricing fix)
Platform	Self-hosted Docker, backup/restore, Mana Pool CSV export

Appendix B — Roadmap phases

Phase	Theme	Key deliverables
5	Polish	Search, pricing persistence, bulk transfer, webhook auth
6	Foundation	Login, SKU ledger, sort/promote, reservations
7	Games & scan	Pokémon, printing picker, price overlay
8	Autopricing	Rules, refresh, per-channel, bulk reprice
9	Channels	Sync, templates, oversell guard
10	Counter	POS, store credit, buylist
11	Shipping	Unified orders, labels, tracking

Appendix C — Amazon 6-pager section guide

Section	Typical length	Purpose
Introduction	½–1 page	Context, complication, resolution; hook the reader
Goals	½ page	Output metrics (results) and input metrics (levers)
Tenets	½ page	Decision principles that resolve trade-offs
State of the Business	~1 page	Data-driven snapshot of current performance and gaps
Lessons Learned	½–1 page	What worked and failed; informs priorities
Strategic Priorities	2–3 pages	Execution plan tied to goals
Appendix	Unlimited	Tables, charts, supporting detail

References: Amazon narrative memo practice as described in industry summaries (e.g. sixpagememo.com, [Maestra.ai](https://maestra.ai)). Main body target: six pages; appendix excluded from page count.